



Лекція2: Проєктування та реалізація ноди керування рухом

КНУБА

Викладач: Маринський Костянтин Володимирович

КУРС «ПРОГРАМУВАННЯ ДИНАМІЧНИХ СИСТЕМ»

Київ – 2026



Мета лекції

Що ви дізнаєтесь сьогодні?

Ця лекція закладає фундамент розуміння того, як реалізується рух робота у середовищі ROS2. Після заняття ви зможете самостійно проєктувати та реалізовувати ноди керування рухом для наземних роботів.

01

Фізика руху

Зрозуміти, як саме рухається двоколісний робот, і яка математика за цим стоїть.

03

Повідомлення **Twist**

Розібрати структуру `geometry_msgs/Twist`: лінійна та кутова складові.

02

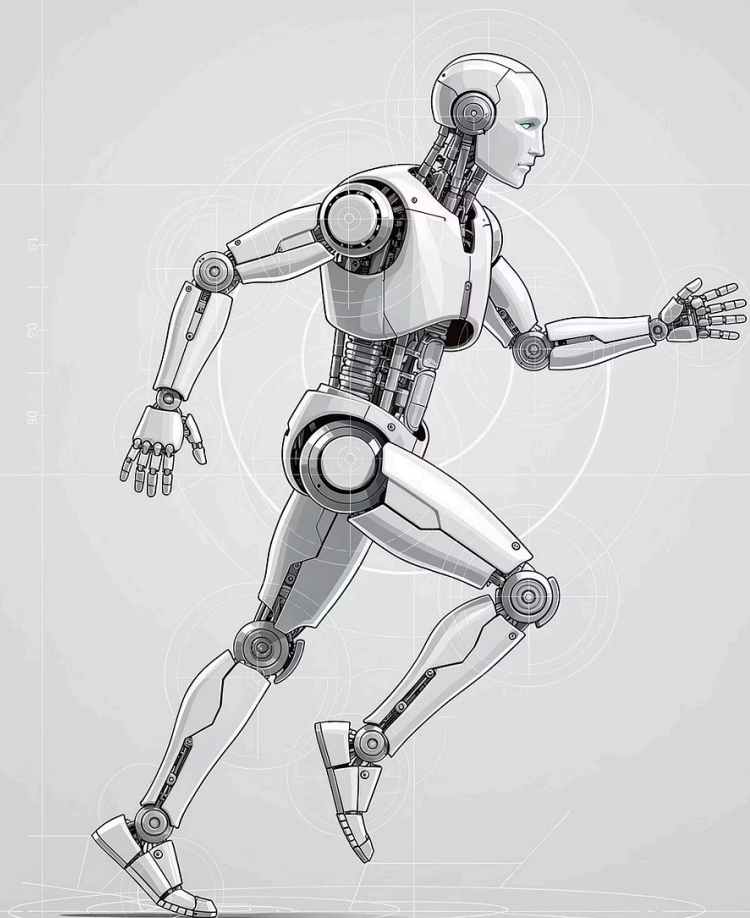
Топік `/cmd_vel`

Вивчити стандартний ROS2-топік для передачі команд швидкості між вузлами.

04

Проєктування ноди

Навчитися проєктувати **Motion Node** з перевіркою значень і обмеженням швидкості.



Зміст лекції

Сьогодні ми крок за кроком розглянемо всі аспекти, необхідні для створення надійного Motion Node.

Основи руху

Механіка та типи швидкості
двоколісних роботів.

ROS2 для керування

Топік `/cmd_vel` та повідомлення
Twist.

Архітектура Motion Node

Проектування та логіка
незалежного вузла.

Захисні механізми

Як уникнути проблем та
підвищити безпеку.

Підсумки

Ключові висновки для вашої практики.

Як робот рухається насправді?

Більшість мобільних роботів, особливо наземних, використовують принцип **диференціального приводу**. Це означає, що кожне колесо (зазвичай два) має свій незалежний двигун. Керуючи швидкістю обертання цих коліс, ми можемо досягти різноманітних рухів – від прямолінійного руху до поворотів на місці.





Як робот рухається насправді?

Автомобіль

- Двигун надає тягову силу
- Кермо задає напрямок
- Газ і гальма контролюють швидкість

Складна механічна система із розподіленим керуванням.

Двоколісний робот

- Лише два незалежних колеса
- Напрямок — через різницю швидкостей
- Без окремого керма або осі повороту

Простіша конструкція, але потребує точного алгоритму керування.

Диференціальний привід

Диференціальний рух — основний принцип керування наземними роботами з двома колесами. Поведінка робота повністю визначається співвідношенням швидкостей лівого та правого колеса.

Прямолінійний рух

Обидва колеса обертаються з **однаковою швидкістю** в одному напрямку. Робот рухається чітко вперед або назад без відхилень.

Поворот

Одне колесо обертається **швидше за інше**. Робот плавно повертає убік колеса з меншою швидкістю — траєкторія дуга.

Обертання на місці

Колеса обертаються у **протилежних напрямках** з рівною швидкістю. Робот крутиться навколо власної осі без переміщення.

 Ключова ідея: уся логіка руху зводиться до двох чисел — швидкостей лівого та правого моторів.



Два типи швидкості в робототехніці

Лінійна швидкість (linear velocity)

Визначає, **наскільки швидко** робот переміщується у просторі вперед або назад вздовж своєї осі. Вимірюється у метрах на секунду (м/с). У ROS2 — це поле `linear.x` повідомлення Twist.

Аналог: спідометр автомобіля — 50 км/год.

Кутова швидкість (angular velocity)

Визначає, **наскільки швидко** робот обертається навколо вертикальної осі. Вимірюється у радіанах на секунду (рад/с). У ROS2 — це поле `angular.z` повідомлення Twist.

Аналог: поворот керма автомобіля вліво або вправо.

Топік /cmd_vel

У ROS2 існує стандартний топік /cmd_vel (скорочення від *command velocity* — команда швидкості). Це «канал зв'язку», через який будь-який модуль системи може надсилати команди руху, а Motion Node їх виконує.



```
graph LR; A[Інтерфейс / Teleop] --> B[/cmd_vel топік]; B --> C[Motion Node];
```

Інтерфейс / Teleop

/cmd_vel топік

Motion Node

Такий підхід дозволяє легко замінювати джерело команд: телеоператор, GUI, алгоритм автономного керування — все використовує один і той самий топік.

Структура повідомлення `geometry_msgs/Twist`

Повідомлення типу `geometry_msgs/msg/Twist` — стандарт ROS2 для передачі команд швидкості. Воно містить шість полів, розбитих на два вектори.

Лінійна складова (`linear`)

- `linear.x` — рух вперед / назад
- `linear.y` — бічний рух (для голономних)
- `linear.z` — вертикальний рух (дрони)

Кутова складова (`angular`)

- `angular.x` — крен (`roll`)
- `angular.y` — тангаж (`pitch`)
- `angular.z` — ристання, поворот (`yaw`)

❏ Для наземного двоколісного робота використовуються лише два поля: `linear.x` (вперед/назад) та `angular.z` (поворот). Решта полів дорівнюють нулю.

Практичне застосування

Приклади команд руху

Розглянемо три базові сценарії керування наземним роботом через `/cmd_vel`. Кожен сценарій — це конкретне поєднання значень `linear.x` та `angular.z`.

1

Рух вперед

```
linear.x = 0.5  
angular.z = 0.0
```

Робот рухається прямо зі швидкістю 0.5 м/с. Без обертання.

2

Поворот праворуч

```
linear.x = 0.0  
angular.z = -0.5
```

Робот обертається праворуч на місці. Від'ємне значення — за годинниковою стрілкою.

3

Обертання на місці

```
linear.x = 0.0  
angular.z = 1.0
```

Робот крутиться навколо своєї осі без переміщення. Додатне значення — проти годинникової стрілки.



Архітектура Motion Node

Motion Node — це центральний вузол керування рухом. Він отримує команди швидкості, обробляє їх і передає до апаратного рівня — драйвера моторів.

1

/cmd_vel

Топік із командами швидкості типу Twist від GUI, телеоператора або автономного агента

2

Motion Node

Підписується на топік, перевіряє значення, обмежує швидкість, логує команди

3

Motor Driver

Апаратний інтерфейс, що перетворює команди у сигнали для фізичних двигунів робота

Чому Motion Node — окремий вузол?

Принцип розподілу відповідальності (Single Responsibility Principle) є фундаментальним у розробці надійних роботизованих систем. Керування рухом не можна «змішувати» з іншою логікою.



Критична відповідальність

Помилка у керуванні рухом може призвести до фізичної шкоди. Ізоляція дозволяє максимально контролювати цей вузол і мінімізувати ризики.



Незалежне тестування

Окремий вузол можна тестувати у симуляторі без камери чи GUI. Це прискорює розробку і полегшує пошук помилок.



Модульність системи

Замінити або оновити Motion Node можна без торкання до інших частин системи. Це ключова перевага мікросервісної архітектури ROS2.

📌 **Принцип архітектури:** «Одна нода — одна відповідальність». Це не просто стиль коду — це вимога до безпечних робототехнічних систем.



Що може піти не так?

Motion Node працює з фізичним обладнанням у реальному часі. Будь-яка помилка або некоректне значення може призвести до аварійної ситуації. Розглянемо типові загрози.

⚠ Неконтрольована швидкість

Якщо вхідне значення `linear.x` не перевіряється, робот може отримати команду рухатись на надмірній швидкості, що призведе до зіткнення або падіння.

⚠ Різкий поворот

Велике значення `angular.z` без обмеження може спричинити різкий розворот, втрату рівноваги або пошкодження механіки.

⚠ Втрата сигналу

Якщо Motion Node перестає отримувати повідомлення з `/cmd_vel`, робот має автоматично зупинитись — а не продовжувати останню команду нескінченно.

Захисні механізми Motion Node

Надійна Motion Node — це не просто «підписатись і передати команду». Вона має активно захищати систему від некоректних вхідних даних і нештатних ситуацій.



Перевірка значень

Кожне вхідне повідомлення Twist перевіряється на наявність NaN, Inf та виходу за допустимий діапазон перед передачею до драйвера.



Обмеження швидкості

Значення `linear.x` та `angular.z` затискаються у безпечних межах (наприклад, `max 1.0` м/с) незалежно від вхідної команди.



Логування команд

Усі отримані команди записуються у лог із часовою міткою. Це дозволяє відтворити та проаналізувати поведінку робота після інциденту.



Аварійна зупинка

При відсутності нових команд протягом визначеного таймауту нода автоматично надсилає нульову швидкість для безпечної зупинки.



Підсумки лекції

Сьогодні ми розглянули повний ланцюг від фізики руху до архітектури програмного вузла. Ці знання є основою для практичної роботи з мобільними роботами у середовищі ROS2.

1 Диференціальний привід

Поведінка двоколісного робота повністю визначається різницею швидкостей лівого та правого колеса.

2 Twist та /cmd_vel

Стандартний спосіб передачі команд руху у ROS2 — топик `/cmd_vel` з повідомленнями `geometry_msgs/Twist`. Для наземного робота — лише `linear.x` та `angular.z`.

3 Single Responsibility

`Motion Node` — ізольований, тестований вузол з єдиною відповідальністю: безпечно передавати команди до моторів.

4 Безпека — перш за все

Перевірка вхідних значень, обмеження швидкості, логування та аварійна зупинка — обов'язкові компоненти будь-якої виробничої `Motion Node`.